



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

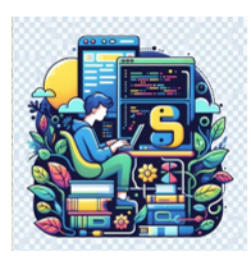
PROGRAMMING WITH PYTHON

Object-Oriented Programming (OOP) and Advanced Concepts

Class and Instance Variables

Lekha A

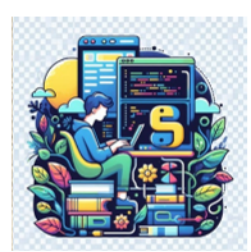
Computer Applications



PROGRAMMING WITH PYTHON

Class and Static Variables

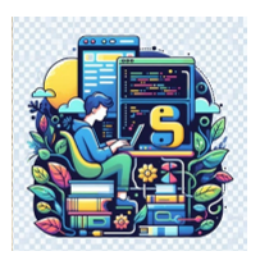
- All objects share class or static variables.
- An instance or non-static variables are different for different objects (every object has a copy).



PROGRAMMING WITH PYTHON

Class Variables

- Class variables are variables that belong to the class itself, rather than to individual instances of the class.
- They are declared inside the class definition, but outside of any method definitions.
- They are typically initialized with a value, just like an instance variable, but they can be accessed and modified through the class itself, rather than through an instance.



PROGRAMMING WITH PYTHON

Class Variables

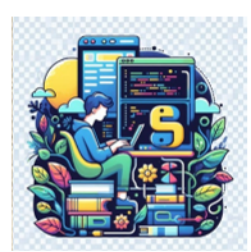
- Class variables are also sometimes referred to as static variables.
- This is because they are not associated with any particular instance of the class.
- They are static in the sense that they exist independently of any particular instance of the class.



PROGRAMMING WITH PYTHON

Class Variables

- Belong to the Class
 - Class variables are shared among all instances of a class.
- Defined Outside Methods
 - They are usually defined within the class but outside of any methods using the `className.variableName` notation.



PROGRAMMING WITH PYTHON

Class Variables

```
class Student:
    # Class variable
    school_name = 'Hogwarts School of Witchcraft and Wizardry '

    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no
```

```
# create first object
s1 = Student('Harry', 10)
# access class variable
print(s1.name, s1.roll_no, Student.school_name)
```

Harry 10 Hogwarts School of Witchcraft and Wizardry

```
# create second object
s2 = Student('Ron', 20)
# access class variable
print(s2.name, s2.roll_no, Student.school_name)
```

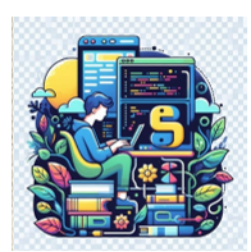
Ron 20 Hogwarts School of Witchcraft and Wizardry



PROGRAMMING WITH PYTHON

Class Variables

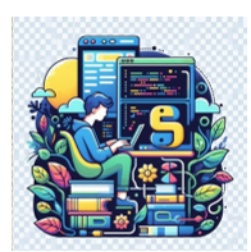
- Shared Across Instances
 - Changes to a class variable are reflected across all instances of the class.
- Typically Used for Constants or Shared Attributes
 - They are commonly used for attributes that are consistent across all instances of the class.



PROGRAMMING WITH PYTHON

Class Variables

- `class_variable` is a class variable shared among all instances
- When the class variable is modified it is reflected on all the objects.



PROGRAMMING WITH PYTHON

Class Variables

```
class MyClass:  
    class_variable = 10  
  
    def __init__(self, value):  
        self.instance_variable = value
```

```
obj1 = MyClass(5)
```

```
obj2 = MyClass(8)
```

```
print(obj1.class_variable)
```

10

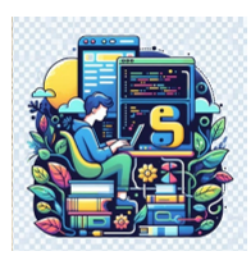
```
print(obj2.class_variable)
```

10

```
MyClass.class_variable = 20
```

```
print(obj1.class_variable)
```

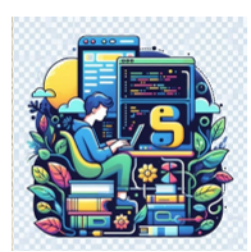
20



PROGRAMMING WITH PYTHON

Accessing Class Variables

- Access inside the constructor by using either self parameter or class name.
- Access class variable inside instance method by using either self of class name
- Access from outside of class by using either object reference or class name.



PROGRAMMING WITH PYTHON

Accessing Class Variables

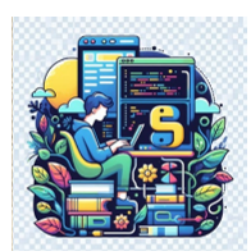
- *##Access inside the constructor by using either self parameter or class name*

```
class Student:
    # Class variable
    school_name = 'Hogwarts School of Witchcraft and Wizardry '

    def __init__(self, name):
        self.name = name
        # access class variable inside constructor using self
        print(self.school_name)
        # access using class name
        print(Student.school_name)
```

```
# create Object
s1 = Student('Sirius')
```

```
Hogwarts School of Witchcraft and Wizardry
Hogwarts School of Witchcraft and Wizardry
```

PROGRAMMING WITH PYTHON

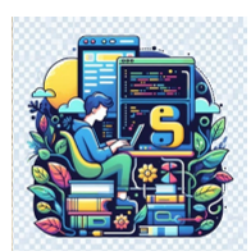
Accessing Class Variables

##Access class variable inside instance method by using either self or class name

```
class Student:
    # Class variable
    school_name = 'Hogwarts School of Witchcraft and Wizardry '

    # constructor
    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no

    # Instance method
    def show(self):
        print('Inside instance method')
        # access using self
        print(self.name, self.roll_no, self.school_name)
        # access using class name
        print(Student.school_name)
```



PROGRAMMING WITH PYTHON

Accessing Class Variables

- # create Object*

```
s1 = Student('Lily', 10)
s1.show()
```

Inside instance method

Lily 10 Hogwarts School of Witchcraft and Wizardry
Hogwarts School of Witchcraft and Wizardry

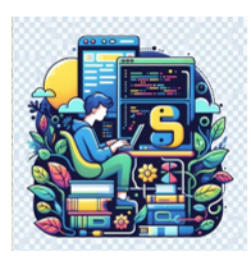
```
print('Outside class')
# access class variable outside class
# access using object reference
print(s1.school_name)
```

Outside class

Hogwarts School of Witchcraft and Wizardry

```
# access using class name
print(Student.school_name)
```

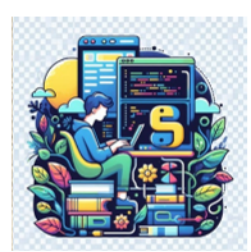
Hogwarts School of Witchcraft and Wizardry



PROGRAMMING WITH PYTHON

Modifying Class Variables

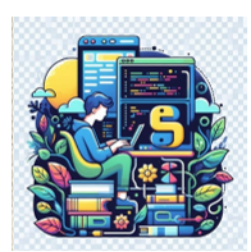
- Generally, value is assigned to a class variable inside the class declaration.
- However, it can be changed either in the class or outside of class.
 - Note: It is recommended to change the class variable's value using the class name.



PROGRAMMING WITH PYTHON

Modifying Class Variables

- If a class variable is modified using an instance, it doesn't change the class variable itself.
- Instead, it creates an instance variable with the same name that shadows (or hides) the class variable for that instance.



PROGRAMMING WITH PYTHON

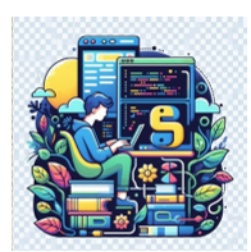
Modifying Class Variables

- `## Modify Class Variables`

```
class Student:
    # Class variable
    school_name = 'Hogwarts School of Witchcraft and Wizardry '

    # constructor
    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no

    # Instance method
    def show(self):
        print(self.name, self.roll_no, Student.school_name)
```



PROGRAMMING WITH PYTHON

Modifying Class Variables

- # create Object*
`s1 = Student('Peter', 10)`
`print('Before')`
`s1.show()`

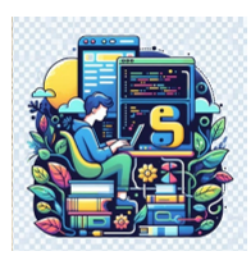
Before

Peter 10 Hogwarts School of Witchcraft and Wizardry

Modify class variable
`Student.school_name = 'Hogwarts'`
`print('After')`
`s1.show()`

After

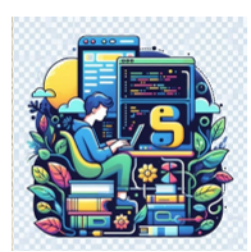
Peter 10 Hogwarts



PROGRAMMING WITH PYTHON

Instance Variables

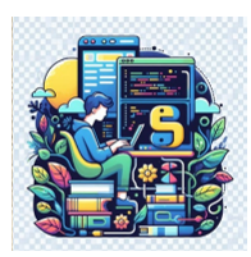
- If the value of a variable varies from object to object, then such variables are called instance variables.
- For every object, a separate copy of the instance variable will be created.
- Instance variables are not shared by objects.



PROGRAMMING WITH PYTHON

Instance Variables

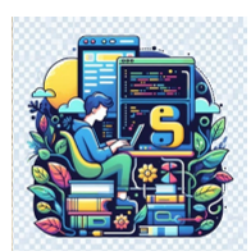
- Every object has its own copy of the instance attribute.
- This means that for each object of a class, the instance variable value is different.
- Instance variables are used within the instance method.



PROGRAMMING WITH PYTHON

Instance Variables

- The instance method is used to perform a set of actions on the data/value provided by the instance variable.
- The instance variable can be accessed using the object and dot (.) operator.

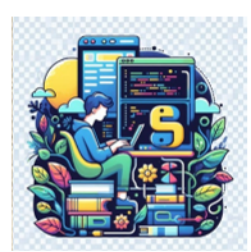


PROGRAMMING WITH PYTHON

Instance Variables

- *##Instance Variable*

```
class Student:  
    # constructor  
    def __init__(self, name, age):  
        # Instance variable  
        self.name = name  
        self.age = age
```



PROGRAMMING WITH PYTHON

Instance Variables

```
# create first object
s1 = Student("Colin", 20)

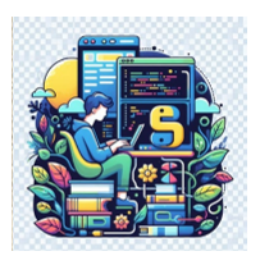
# access instance variable
print('Object 1')
print('Name:', s1.name)
print('Age:', s1.age)
```

Object 1
Name: Colin
Age: 20

```
# create second object
s2 = Student("Hermione", 10)

# access instance variable
print('Object 2')
print('Name:', s2.name)
print('Age:', s2.age)
```

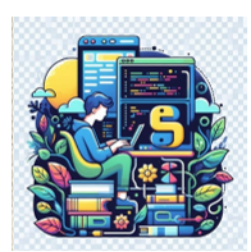
Object 2
Name: Hermione
Age: 10



PROGRAMMING WITH PYTHON

Modify Instance Variables

- Modify the value of the instance variable and assign a new value to it using the object reference.
- When the instance variable's values of one object is changed, the changes will not be reflected in the remaining objects because every object maintains a separate copy of the instance variable.



PROGRAMMING WITH PYTHON

Modify Instance Variables

```
# create object  
stud = Student("Draco", 20)  
  
print('Before')  
print('Name:', stud.name, 'Age:', stud.age)
```

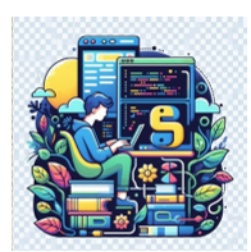
Before

Name: Draco Age: 20

```
# modify instance variable  
stud.name = 'Malfoy'  
stud.age = 15  
  
print('After')  
print('Name:', stud.name, 'Age:', stud.age)
```

After

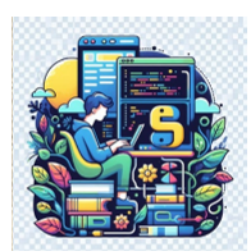
Name: Malfoy Age: 15



PROGRAMMING WITH PYTHON

Dynamically creating Instance Variables

- Instance variables can be created from the outside of class to a particular object.
- To add the new instance variable to the object.
 - `object_referance.variable_name = value`



PROGRAMMING WITH PYTHON

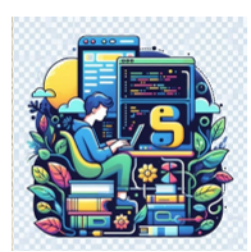
Dynamically creating Instance Variables

- *##Dynamic Creation of Instance Variable*

```
class Student:
    def __init__(self, name, age):
        # Instance variable
        self.name = name
        self.age = age
```

```
# create object
stud = Student("George", 20)
```

```
print('Before')
print('Name:', stud.name, 'Age:', stud.age)
```



PROGRAMMING WITH PYTHON

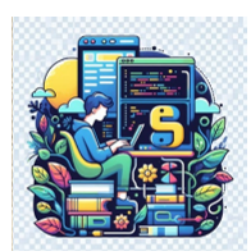
Dynamically creating Instance Variables

- ```
add new instance variable 'marks' to stud
stud.marks = 75
print('After')
print('Name:', stud.name, 'Age:', stud.age, 'Marks:', stud.marks)
```

After

Name: George Age: 20 Marks: 75





# PROGRAMMING WITH PYTHON

## Dynamically creating Instance Variables

```
create object
stud = Student("Draco", 20)
```

```
print('Before')
print('Name:', stud.name, 'Age:', stud.age)
```

Before  
Name: Draco Age: 20

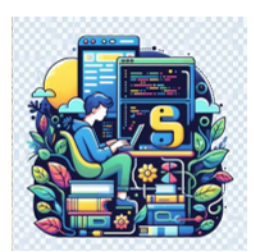
```
print('After')
print('Name:', stud.name, 'Age:', stud.age, 'Marks:', stud.marks)
```

After

```

AttributeError Traceback (most recent call last)
Cell In[32], line 2
 1 print('After')
----> 2 print('Name:', stud.name, 'Age:', stud.age, 'Marks:', stud.marks)

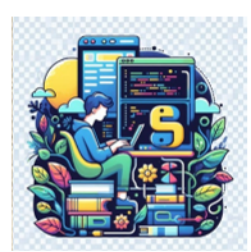
AttributeError: 'Student' object has no attribute 'marks'
```



# PROGRAMMING WITH PYTHON

## Instance vs. Class Variables

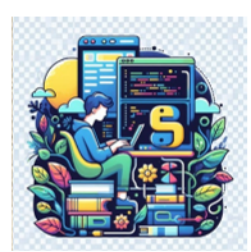
| • Instance Variable                                                                                   | Class Variable                                                                                                     |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Instance variables are not shared by objects. Every object has its own copy of the instance attribute | Class variables are shared by all instances.                                                                       |
| Instance variables are declared inside the constructor i.e., the <code>__init__()</code> method.      | Class variables are declared inside the class definition but outside any of the instance methods and constructors. |



# PROGRAMMING WITH PYTHON

## Instance vs. Class Variables

| • Instance Variable                                                                    | Class Variable                                                  |
|----------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| It gets created when an instance of the class is created.                              | It is created when the program begins to execute.               |
| Changes made to these variables through one object will not reflect in another object. | Changes made in the class variable will reflect in all objects. |



# PROGRAMMING WITH PYTHON

## Recap

- Class Variables
- Accessing Class Variables
- Modify Class Variables
- Instance Variables
- Modify Instance Variables
- Class vs Instance Variables
- Dynamically creating Instance Variables



**THANK YOU**

---

**Lekha A**  
Computer Applications